

Multi-Bottleneck Fairness in RDMA Fabrics: Measurement and an Explicit-Rate Service Mode

How deployable RDMA congestion control behaves under multi-bottleneck contention, why sender-side correction stalls, and RDMA-RCP over in-network telemetry

Haoyu Wang

University of Massachusetts Boston
Boston, MA, USA
haoyu.wang001@umb.edu

Zerin Shaima Meem

University of Nebraska Omaha
Omaha, NE, USA
zmeem@unomaha.edu

Bo Sheng

University of Massachusetts Boston
Boston, MA, USA
bo.sheng@umb.edu

Xiaoqian Zhang

University of Nebraska Omaha
Omaha, NE, USA
xiaoqianzhang@unomaha.edu

Abstract

Modern datacenter traffic increasingly crosses several contended links in series, yet deployable RDMA congestion control is evaluated almost exclusively on single-bottleneck benchmarks. We present an empirical study of multi-bottleneck fairness on lossless RDMA fabrics. On a parameterized parking-lot benchmark with a fluid max-min oracle, *every* deployable scheme we test is unfair to the multi-bottleneck flow — DCQCN 1.7–2.3×, TIMELY 1.8–2.0×, DCTCP 1.6–1.8×, HPCC 1.9–2.0× — growing with bottleneck count and robust to start-time jitter. For HPCC we localize the mechanism: a winner-take-all equilibrium in which the multi-bottleneck flow collapses to ~0.4 Gbps. Six sender-only corrections fail, including a sender-side mirror of RCP driven by the same INT signals — evidence that the obstacle is not information but *consistency*: max-min needs one shared reference rate per link, and private per-sender estimates never agree. Placing that reference at the switch — RDMA-RCP, an RCP service mode carried in one 64-bit INT field with one fair-rate register (three words of state) per port — restores the max-min allocation (penalty $\approx 1.005\times$) with zero PFC, rate-only or with a canonical $cwnd = R \cdot RTT$ window, and cuts allreduce-style coflow completion time by ~21% at every ECMP seed. We also measure the deployment boundary: RCP-mode and HPCC traffic do not coexist fairly on a shared link, even under a weighted-DRR reservation, because both classes’ control signals remain port-global; we therefore position RDMA-RCP as a *fabric-wide* (or isolated-service) mode. Stock HPCC’s code path remains byte-for-byte unchanged.

1 Introduction

HPCC [15] and its derivatives represent the state of the art in datacenter RDMA congestion control: the switch stamps raw per-hop state into INT headers, and the sender computes a precise utilization estimate toward a target η . On the canonical single-bottleneck benchmarks HPCC matches or beats DCQCN, TIMELY, and DCTCP on flow completion time and queueing — and that is where deployable RDMA congestion control is, almost exclusively, evaluated.

Modern datacenter traffic, however, increasingly crosses *several contended links in series*: cross-pod ML collectives, disaggregated memory and storage, multi-tenant aggregation and core layers. The practical stake is concrete: a collective completes when its *slowest* flow completes, and the slowest flows are precisely the cross-fabric, multi-bottleneck ones. On a 16-flow ring allreduce over a $k=4$ fat-tree, we measure stock HPCC inflating job completion time by ~27% over what max-min allocation delivers — at every ECMP hash seed we test (§5.4). Whether deployable RDMA congestion control is fair to such traffic has not been carefully measured. This paper measures it.

Max-min fairness is a policy choice, not a universal congestion-control objective; our claims target traffic classes whose completion is gated by multi-bottleneck flows, and §5.14 delimits where it is the wrong target. We proceed with a deterministic, oracle-grounded methodology. Our simulator is the original HPCC authors’ ns-3 code base, upgraded to a tagged modern release (ns-3.45): the changes are interface-level — build-system, header, and deprecated-API adaptations — while the congestion-control logic, switch datapath, and wire formats are the original code (one latent field-initialization bug, exposed only by the 376-node topology, is fixed and documented). We validated the upgrade by reproducing the original paper’s baseline comparisons across five schemes and two production workloads (§5); the reproduction record and raw results are public. The measurements here are therefore produced by the original HPCC implementation’s logic, not a re-implementation. Our findings:

- (1) **The failure spans every scheme we evaluate, and we characterize HPCC’s instance** (§2, §5.3). Every deployable scheme we test is unfair to the multi-bottleneck flow — DCQCN 1.70–2.32×, TIMELY 1.83–2.01×, DCTCP 1.61–1.78×, HPCC 1.90–1.96× at $N = 2-4$ — all growing with bottleneck count, and robust to start-time jitter. For HPCC we isolate the mechanism: the failure is *winner-take-all* (the long flow collapses to ~0.4 Gbps against ~23 Gbps competitors; 3.5× for small flows), RTT-independent, immune to HPCC’s own per-hop multi-rate mode and to INT compression (PINT), and compounded past four bottlenecks by a

wire-format cap ($\text{maxHop} = 5$) that corrupts the control input. A single-bottleneck control ($N=1$: 1.10–1.39× pairwise spread, direction set by arrival order) separates HPCC’s baseline rate lottery from the systematic multi-bottleneck penalty on top.

- (2) **Sender-side control cannot close the gap — and the reason is consistency, not information** (§3). We evaluate six sender-only designs: five heuristics, plus the strongest possible variant — a full sender-side mirror of the RCP recursion driven by the same INT-visible inputs the switch would use. The mirror still fails (1.56× even with synchronized starts; 1.65×/0.81× under staggered arrival, the *direction* set by arrival order), while the identical law at the switch holds $\approx 1.0\times$. Max-min requires all flows at a link to adopt *one shared reference rate*; private multiplicative estimates preserve their initial disagreement. We present this as controlled empirical evidence, not a formal impossibility.
- (3) **An RCP service mode over INT restores max-min, verified end to end** (§4, §5). RDMA-RCP carries a switch-computed RCP fair rate in one 64-bit INT field with one fair-rate register (three words of state) per egress port, alongside byte-for-byte unchanged stock HPCC. It reaches penalty $\approx 1.005\times$ across bottleneck counts, asymmetric sizes, staggered and jittered starts, and ECMP hash seeds, with zero PFC — operating rate-only or with a canonical RCP window ($\text{cwnd} = R \cdot \text{baseRTT}$), and cutting allreduce coflow JCT by $\sim 21\%$ at every seed. We equally report its deployment boundary: sharing a link with stock HPCC serves neither class, even under a weighted-DRR reservation, because both controllers read port-global telemetry (§4.3) — so we position RDMA-RCP as a fabric-wide or isolated-service mode. The mechanism is RCP — we claim the measurement study and the placement result, not a new control law.

Artifacts. The implementation, the topology/workload generators, the max-min oracle, and deterministic scripts that regenerate the figures (`make_figures.py`) and the ablation/sensitivity/baseline tables (`run_sensitivity.py`) from source are available in our artifact-review repository.¹

2 Background and Motivation

2.1 HPCC, briefly

HPCC’s per-hop utilization estimate for hop i is

$$u_i = \frac{\text{txRate}_i}{C_i} + \frac{\min(\text{qlen}_i, \text{qlen}_i^{\text{prev}}) \cdot \text{maxRate}}{C_i \cdot W} \quad (1)$$

where the first term reflects bandwidth utilization and the second is a queue-drain term that keeps standing queues bounded. In single-rate mode the sender takes $U = \max_i u_i$, smooths it over the base RTT, and applies

$$\text{new_rate} = \begin{cases} \text{curRate}/\text{max } c + R_{AI} & \text{max } c \geq 1 \\ \text{curRate} + R_{AI} & \text{otherwise} \end{cases}$$

with $\text{max } c = U/\eta$. The window W is sized to $m_{\text{win}} \cdot \text{rate}/\text{NIC_line}$ under `var_win`.

¹<https://github.com/UNOCourseDemo/hpcc-fs>

2.2 The multi-bottleneck parking lot

The *parking lot* is a classic congestion-control fairness benchmark [6, 12]. The name is an analogy: a long boulevard (the long flow’s path) is crossed by a series of side streets, and at each intersection a different car (a single-link cross flow) merges on and exits. Because every link is equally contended, the *max-min fair* allocation [6] gives every flow an equal share, so the long flow should run exactly as fast as each cross flow despite traversing many contention points rather than one. The parking lot therefore isolates the multi-bottleneck fairness question in its simplest form, with no confounding asymmetry in link capacity or hop count of the competitors. One control is essential before reading any deviation as a multi-bottleneck effect: on a *single* shared bottleneck ($N=1$) with two symmetric 50 MB flows, stock HPCC already allocates unequally (37.20 vs 26.72 ms, a 1.39× spread), while RDMA-RCP equalizes them exactly (33.89 vs 33.89 ms). Stock HPCC therefore carries a baseline rate unfairness between symmetric flows; the multi-bottleneck penalty we characterize is the *additional* deviation on top of it, growing to 1.90–1.96× at $N = 2$ –4.

We construct a parameterized parking-lot benchmark with N inter-switch bottleneck links (25 Gbps each) and 100 Gbps host links (Figure 1). One “long” flow traverses all N bottlenecks; one “cross” flow traverses each single bottleneck. Flows are 50 MB by default and start simultaneously at $t = 0$. With two flows sharing each 25 Gbps link, the max-min fair rate is 12.5 Gbps for every flow. All experiments are deterministic: each configuration produces byte-identical artifacts across repeated runs (verified).

Metric. We compare each flow’s actual flow completion time (FCT) against an idealized **fluid max-min oracle** — a continuous bandwidth-sharing model with no protocol dynamics (no slow start, no queueing, no convergence delay) in which the bandwidth is allocated max-min fairly at every instant. We compute the allocation by *progressive filling* [6]: all active flows raise their rate in lockstep until some link saturates; the flows crossing that link are frozen at their current rate; and the process repeats on the remaining flows until every flow is frozen. This is the textbook algorithm for max-min fair rates. Because flows complete and depart over time, we run it *event-driven* — recomputing the allocation at each flow completion — so the oracle FCT correctly accounts for the bandwidth freed as short flows finish. The oracle is therefore the best FCT any rate-allocation scheme could achieve under max-min fairness, and is exact for our fluid topologies. The headline metric is the **unfairness ratio** $\text{max}_{\text{path}} \text{FCT} / \text{mean}(\text{short-path FCT})$; the oracle’s value is 1.0 for equal-size, simultaneous flows.

2.3 Findings F1–F4: characterizing the gap

Magnitude. Stock HPCC’s unfairness ratio at $N \in \{2, 3, 4\}$ is 1.90, 1.92, 1.96: the multi-bottleneck flow’s FCT is roughly twice that of single-bottleneck competitors. **There is a non-monotonic regime change at $N \geq 5$** , where the long flow becomes favored (0.51×); this turns out to be an INT-overflow artifact at $\text{maxHop} = 5$ and we exclude $N \geq 5$ from the rest of this paper’s stock comparisons (Figure 2).²

²The 5-hop cap is HPCC’s original wire format, not an artifact of our ns-3 upgrade: the reference simulator hard-codes $\text{maxHop} = 5$ in its INT header [2], consistent with the paper’s 42-byte worst-case INT overhead (five 8-byte hop records). Conversely, a

Parking lot at $N=4$: one LONG flow + one CROSS flow per bottleneck

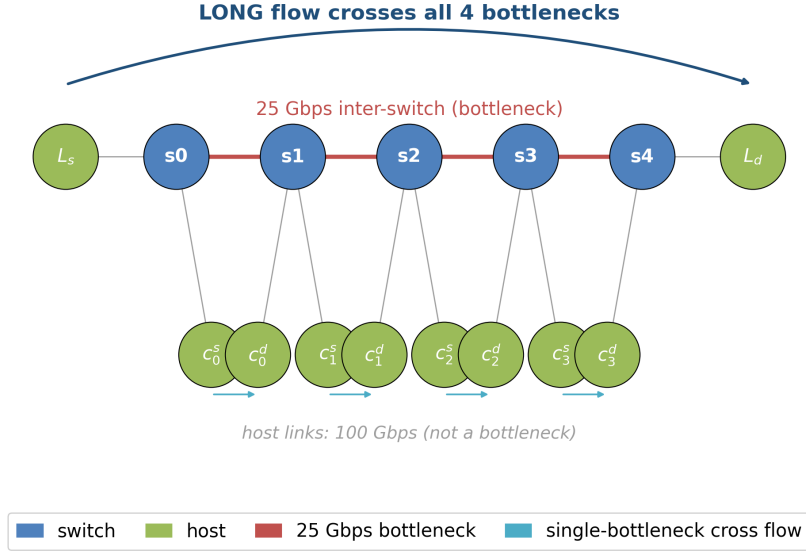


Figure 1: Parking-lot benchmark at $N = 4$. The long flow $L_s \rightarrow L_d$ crosses every bottleneck; each cross flow $c_i^s \rightarrow c_i^d$ traverses exactly one. Host links are not bottlenecks.

Robustness. Size and start-time perturbations give $1.81\text{--}3.53\times$ (Figure 4), worst ($3.5\times$) when the multi-bottleneck flow is small — the latency-sensitive RPC case. *Multi-rate HPCC* (per-hop EWMA, $\min_i R c_i$) helps but does not fix it: $1.44/1.50/1.75\times$ at $N = 2/3/4$, still growing with N . *RTT-equalization rules out RTT bias*: the penalty stays $\sim 1.9\times$ across a $4.5\times$ sweep of cross-flow RTTs, including cross flows with up to $1.8\times$ the long flow’s RTT — this is not RTT unfairness.

Trace diagnosis. At $N = 4$ (Figure 5 left) the long flow collapses to ~ 0.38 Gbps while the four cross flows hold ~ 23.2 Gbps each for the entire 19 ms contention window, with zero PFC and zero loss — *winner-take-all*: at every shared link the local single-bottleneck flow wins and the long flow is squeezed. This is an equilibrium, not a transient: with 500 MB flows ($10\times$ the contention, $\sim 9,000$ RTTs) the ratio is $1.975\times$ and the long flow’s rate is flat at 0.39 Gbps from 20 ms to 170 ms.

3 The Limits of Sender-Side Control

A natural first design instinct — and the one our project explored exhaustively — is to keep all state and computation at the sender, since HPCC’s value proposition is its stateless switch design. We evaluated six sender-only correction variants behind a new `mb_mode` config knob, on top of unchanged HPCC; the `mb_mode = 0` path is byte-identical to stock HPCC (verified). Five are small heuristic changes to `UpdateRateHp`; the sixth is the strongest design the endpoint admits — a full sender-side mirror of the RCP recursion.

parking-lot path at $N \leq 4$ traverses at most five switches — within the INT budget — so the retained $N = 2\text{--}4$ measurements are structurally free of the overflow.

3.1 Five heuristic variants

The five heuristics span the sender’s design space: a k -aware additive boost scaled by the congested-hop count ($1.63\times$, the best case, at $\gamma=50$); a debiased max/mean blend of per-hop u_i ($1.95\times$ — per-hop u_i are nearly equal here, no max-bias to debias); a responsibility-weighted multiplicative decrease ($1.89\times$ — the MD branch barely fires at η -hold); a k -th-root MD ($1.92\times$); and a share-aware additive increase on every flow ($1.71\times$). Stock is $1.96\times$; the target is 1.0.

3.2 Isolating placement: identical observations, private vs. shared state

We first isolate placement cleanly: a deterministic fluid model of one link where every controller receives the exact same (y, q) observations at the same update instants, with identical rule, gains, and initialization — only the *location of the recursion state* differs (`run_consistency.py`; flow 2 arrives at $t=5$ ms). Shared register: the late arrival inherits the evolved state; $12.5/12.5$ Gbps. Private, synchronized: identical trajectories, equally fair — the information is sufficient. Private, staggered: the newcomer initializes fresh, both multiply by the same factor thereafter, and the ratio freezes at exactly 2:1 ($16.67/8.33$ Gbps) forever. The one thing a sender cannot replicate is the register’s *history*.

3.3 The full stack: a virtual-RCP mirror

The fluid model isolates the mechanism; `mb_mode 6` confirms it in the full stack. RCP’s inputs — C, y, q — are all sender-visible in HPCC’s INT record, so the sender runs the switch’s recursion *privately*: per hop, a virtual fair rate $\hat{R}_i$ updated by the identical rule

Table 1: Sender-side virtual RCP (mb_mode 6) vs. the switch mode: same control law, same INT-visible inputs, different placement. $N=4$ parking lot; “late” = that side starts +5 ms after the other.

	synchronized	long late	crosses late
switch (shared per-port R)	1.005×	1.008×	1.003×
sender (private $\hat{R}$)	1.560×	1.654×	0.813×

from the flow’s own INT samples, sending at $\min_i \hat{R}_i$ (same gains, same $R_0 = C/2$, the sender’s own base RTT as control interval). Beyond the fluid model’s arrival asymmetry, the real stack adds per-flow sampling and feedback-delay asymmetry — which is why the mirror falls short even under synchronized starts.

Table 1: the mirror reaches only 1.56×

3.4 The structural reason: consistency, not information

Two observations explain the pattern. *First, the η -hold equilibrium:* once the contended links settle at target utilization η , HPCC’s multiplicative term $1/\max c \approx 1$ and every flow holds its current rate — whoever grabbed more at startup keeps it. Additive nudges (modes 1, 5) are orders of magnitude too slow relative to flow lifetimes (our 200 ms runs are $\sim 10^4$ RTTs and they never converge), and MD-side modifications (modes 3, 4) rarely fire at the hold point.

Second — what mode 6 isolates — max-min requires a shared reference. The information needed to compute the fair share is sender-visible; what the sender cannot obtain is *agreement*. The max-min allocation is reached when every flow at a bottleneck adopts the *same* rate. Private per-sender estimates, updated multiplicatively from asynchronous samples, preserve their initial disagreement instead of contracting it; a per-port register at the switch is the cheapest consistent shared reference — every packet reads the same R , so a newcomer adopts the incumbents’ rate within one RTT. This is why RCP [9, 10] and XCP [12] place the computation in the network. Our contribution is the controlled demonstration that, for INT-style RDMA control, the *placement* is what matters: six sender-side designs spanning heuristics to a full RCP mirror fail, while the identical law at the switch succeeds. We present this as strong empirical evidence, not a formal impossibility result.

4 RDMA-RCP Design

The empirical and structural arguments of §3 motivate a design that (a) makes the *dominant* flows yield rather than merely nudging the victims, and (b) places a fair-rate *signal* at the switch, since the sender cannot recover the fair share from a load signal alone. We do not have the switch count flows and divide; instead each port runs a standard RCP control law on aggregate load and queue (below) whose fixed point is the max-min fair rate C/N — the switch converges to the fair share without ever materializing N . We take the most minimal mechanism consistent with these constraints,

packaged as an *operating mode* selected per traffic class alongside stock HPCC on the same INT transport (§4.3).

Background: RCP. The Rate Control Protocol (RCP) [9, 10] is a router-assisted congestion control in which each link advertises a *single* rate R to *all* flows crossing it, holding no per-flow state. The key point is that R is not an aggregate budget for the link; it is the rate *each individual flow* should send at, such that when all of them do, the link is exactly full and shared equally. A router raises R when its link has spare capacity and lowers it when a queue forms; routers stamp R into passing packets, each flow sends at the minimum R along its path, and that rate is echoed to the source. A single flow adopting R therefore does not seize the link — it takes its share, because R has already been driven down by the load the other flows place on the same port. Concretely, with N flows each sending R the offered load is NR , and the controller drives $NR \rightarrow C$ (idle link $\Rightarrow R$ rises; standing queue $\Rightarrow R$ falls), so at equilibrium $R \approx C/N$ — the max-min (processor-sharing) fair share, tracked *without ever counting N* : as flows arrive or leave, R self-adjusts (a lone flow’s R rises to C ; a third arrival drops everyone to $C/3$). RCP thus supplies exactly the missing ingredient from §3: a fair-share signal computed where the flow count is implicitly observable (the link), not at the sender. RDMA-RCP runs this control law once per egress port and carries its R in the INT header; a flow’s path-minimum R approximates network-wide max-min fairness (within 1.2% of the oracle on our benchmarks), since a flow bottlenecked elsewhere contributes less load here and the loop hands the slack to the rest.

4.1 The mechanism

The entire protocol is Algorithms 1 and 2. The switch maintains one scalar fair rate per egress port and stamps the path-minimum into a single header field; the sender reads that field from the returning ACK and adopts it. No per-flow switch state, no per-hop record, no receiver changes.

At the switch (Alg. 1). Each egress port keeps a single scalar fair rate R_j , refreshed once per control interval d — a fabric-wide constant set to the maximum base RTT, $21.4 \mu\text{s}$ in all our experiments — by the standard RCP rule $R \leftarrow R(1 + (\alpha(C - y) - \beta q/d)/C)$, where y is the load observed over the last interval and q the queue backlog. The term $\alpha(C - y)$ chases spare capacity (positive while the link is under-utilized, pushing R up) and $\beta q/d$ brakes when a standing queue forms (pulling R down); they balance at $y \approx C$, $q \approx 0$, which is the $R \approx C/N$ fixed point above. We use the RCP defaults $\alpha = 0.4$, $\beta = 0.226$ (untuned; the result is insensitive to them, §5.9). The port’s entire state is three numbers — R_j , the last update time, and a byte-counter snapshot — with *no* per-flow data.

On the wire. The FS INT mode (IntHeader::FS) carries one 64-bit field, `fairRate`; the serialized cost is 8 bytes per packet *independent of hop count* (vs. 8 bytes *per hop* for stock HPCC). Each switch min-aggregates R_j into this field (first hop initialises it), so the receiver sees the path-minimum fair rate and echoes the header back unchanged — no receiver changes.

At the sender (Alg. 2). The new ACK handler `HandleAckFs` reads `fairRate` from the returning ACK and sets the sending rate to it,

Algorithm 1 RDMA-RCP switch — per egress port j

```

1: State  $R_j$  (fair rate),  $t_j$  (last update),  $B_j$  (txByte snapshot)
2: Const  $C_j$  link capacity;  $\alpha=0.4$ ,  $\beta=0.226$ ;  $d$  control interval ( $\approx$ 
   RTT);  $R_{\min}$ 
3: procedure ONDEQUEUE( $pkt, j$ )  $\triangleright$  every packet leaving port  $j$ 
4:   if  $R_j$  uninitialised then
5:      $R_j \leftarrow C_j/2$ ;  $t_j \leftarrow now$ ;  $B_j \leftarrow txBytes_j$   $\triangleright$  conservative
       start
6:   end if
7:   if  $now - t_j \geq d$  then  $\triangleright$  one control interval elapsed
8:      $y \leftarrow (txBytes_j - B_j) \cdot 8 / (now - t_j)$   $\triangleright$  observed load
9:      $q \leftarrow queueBytes_j \cdot 8$   $\triangleright$  backlog (bits)
10:     $R_j \leftarrow R_j \cdot (1 + (\alpha(C_j - y) - \beta q/d)/C_j)$   $\triangleright$  RCP update
11:     $R_j \leftarrow \text{clamp}(R_j, R_{\min}, C_j)$ 
12:     $t_j \leftarrow now$ ;  $B_j \leftarrow txBytes_j$ 
13:  end if
14:  if  $pkt.fairRate = 0$  then  $\triangleright$  first hop on the path
15:     $pkt.fairRate \leftarrow R_j$ 
16:  else
17:     $pkt.fairRate \leftarrow \min(pkt.fairRate, R_j)$   $\triangleright$  path-min
18:  end if
19:   $txBytes_j \leftarrow txBytes_j + \text{sizeof}(pkt)$ 
20: end procedure

```

Algorithm 2 RDMA-RCP sender (receiver echoes the header unchanged)

```

1: on QP-create:  $rate \leftarrow R_{\min}$   $\triangleright$  conservative start
2: procedure ONACK( $a$ )
3:   if  $a.fairRate = 0$  then return  $\triangleright$  no signal yet
4:   end if
5:    $rate \leftarrow \text{clamp}(a.fairRate, R_{\min}, NICrate)$   $\triangleright$  adopt path-min
       fair share
6: end procedure

```

clamped to $[R_{\min}, NICrate]$. That single assignment is the whole sender control loop.

4.2 Startup smoothing

Two cold-start details: the switch initializes $R = C/2$ (not C , else the first stamp is line rate), and the `cc_mode 11` sender starts at `m_minRate` so the first RTT does not blast before any ACK returns. A flow sees a path-complete fair-rate signal in one RTT; full convergence takes several control intervals (§5: within 5% of fair share by ~ 0.6 ms, ~ 30 RTTs at our $21 \mu\text{s}$ base RTT; within 1% by 2.3 ms) — fast and path-length-independent, though not literally one-RTT.

4.3 Additive integration and coexistence

RDMA-RCP is `cc_mode == 11`. The mode-3 (HPCC) and mode-10 (PINT) wire formats and switch/sender code paths are completely untouched; we add a fourth case to (a) `IntHeader::Mode`, (b) its serialization, (c) the switch’s mode-dispatch, (d) the sender’s mode-dispatch, and (e) one branch in the validation driver. The driver also forces the QP window to 0 in FS mode (rate-only control),

since `var_win` sizes the window from the NIC line rate rather than the bottleneck path BDP and would otherwise cap the multi-hop flow’s effective window well below what is needed at its fair rate. In the shipped `cc_mode 11` the INT wire format is a global choice — one mode per fabric. To test per-class deployment we added `cc_mode 12`: stock HPCC’s 42-byte wire layout (stock packets byte-for-byte unchanged) with per-packet class dispatch; the FS class reuses the record’s first eight bytes and pays stock’s full header cost, conservative for our claim. Both regression gates hold (all-stock $\equiv$ `cc_mode 3` exactly; all-FS $\equiv$ `cc_mode 11`’s $1.005\times$, zero PFC).

Coexistence is not free (negative result). On the $N=4$ parking lot with the long flow in the RCP class and the crosses in the stock class, *neither class gets its fair share*: rate-only RCP overshoots (21.6 vs 12.5 Gbps fair, squeezing stock to 11.1; $0.51\times$ — starvation reverses), the windowed variant under-shoots (6.2 Gbps, $2.53\times$), and a separate priority queue does not help (HPCC keeps its queue near zero and forfeits round-robin turns). This is the paper’s theme inverted: RCP’s fixed point presumes every flow on the port obeys R ; non-compliant flows are invisible to the inference exactly as flow count is invisible to a sender.

We then tested the obvious remedy: a weighted-DRR egress scheduler (additive `dwr_r_weights`; byte-deficit DRR over the non-priority queues) reserving each class its exact 50% per-link entitlement. *It changes nothing* ($0.511\times$): the scheduler never binds, because stock HPCC’s INT still reports *port-global* bytes and queue — the stock class sees $u \approx 1$ from RCP traffic, holds low, and never offers its reserved share, which RCP then absorbs. A fair scheduler cannot help a class whose controller will not offer its share; coexistence requires the reservation to extend into the *telemetry* — per-class C, y, q — i.e. true slicing, or a separate fabric. All variants ship with the artifact (`run_mixed_mode.py`); PFC is zero throughout.

5 Evaluation

5.1 Methodology

All experiments use the deterministic parking-lot benchmark of §2, driven by our `hpcc-validation`. `cc`. Flows are 50 MB unless noted; `sim` time is 200 ms; the optimized binary is used throughout. We compare against stock HPCC (`cc_mode 3, multi_rate = false`), stock HPCC multi-rate, HPCC-PINT, and the `mb_mode` ablation suite. The simulator is the original HPCC ns-3 code base upgraded to a tagged ns-3.45 release — interface-level changes only; control logic and wire formats unchanged — verified by reproducing the original paper’s baseline comparisons (HPCC vs. DCQCN, TIMELY, DCTCP, and PINT across WebSearch and FB-Hadoop workloads at 30–70% load on a 320-server fat-tree). The full reproduction record — per-run configurations, results, and analysis — ships with the artifact,³ and its verdict is that the upgraded code matches the original paper’s central findings. Metrics, used consistently below: *unfairness ratio* = long-flow FCT / mean cross-flow FCT (oracle = 1.0); *oracle-normalized penalty* = per-flow FCT / fluid max-min

³Reproduction record and methodology: <https://github.com/UNOCourseDemo/hpcc-fs/tree/main/examples/hpcc/repro-verification>. The raw per-run results (554 MB compressed) are archived at https://1drv.ms/f/c/6052297178cce52b/IgAsSQ7gNfdhQrNejod27CpkAYQs8xZs4QtGc_KKVWVZjQLs.

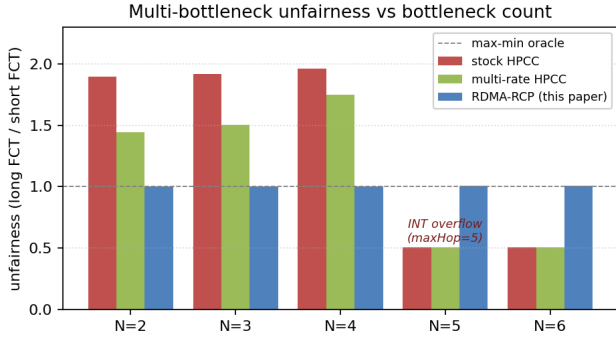


Figure 2: Multi-bottleneck unfairness vs bottleneck count N . RDMA-RCP is flat ≈ 1.0 across the entire sweep; stock and multi-rate HPCC fail at $N \leq 4$ and break at $N \geq 5$ (INT-overflow regime).

oracle FCT; *generalized unfairness* (heterogeneous cases) = worst-long / best-flow penalty; *raw long/short ratio* = mean long / mean short FCT (ECMP study); *slowdown* = FCT / same-size standalone FCT; *coflow JCT* = max FCT over a coflow.

5.2 Headline: N-sweep

Stock and multi-rate HPCC are starved across $N = 2-4$ (penalty $1.90-1.96\times$ for stock, $1.44-1.75\times$ for multi-rate), then break entirely at $N \geq 5$, where the per-hop INT record overflows the maxHop cap (the apparent $0.51\times$ there is the overflow artifact, not a real allocation). RDMA-RCP is flat at $1.003-1.007\times$ across the entire sweep — hop-count-independent by construction, since its single fair-rate field does not grow with path length. HPCC-PINT, which carries one compressed utilization field, inherits HPCC’s load-based control and so still suffers the same $\sim 1.8\times$ ($1.77-1.88\times$) multi-bottleneck penalty — confirming that the fix requires a fair-share signal, not merely a smaller INT footprint. All RDMA-RCP runs have zero PFC events.

5.3 The failure spans our evaluated stack

Is multi-bottleneck unfairness HPCC-specific? We ran every deployable RDMA scheme in our stack on the same benchmark, with harness parity to the original HPCC evaluation (window-based schemes keep their window; DCQCN, TIMELY, and DCTCP run windowless exactly as in their reference configurations). Table 2: every scheme *in our evaluated stack* is unfair to the multi-bottleneck flow and worsens with bottleneck count — DCQCN worst at $N=4$ ($2.32\times$). HPCC is the norm here, not an outlier; only the RCP mode reaches max-min, and only HPCC and the RCP mode hold zero PFC. We do not extend the claim to untested controllers (PowerTCP, Bolt, On-Ramp, Swift), whose feedback models differ materially; a same-benchmark comparison is future work.

5.4 When max-min is the objective: allreduce-style coflow JCT

Collective communication motivates multi-bottleneck fairness: a step completes only when its *slowest* flow completes (JCT = max

Table 2: Unfairness ratio and PFC pauses on the parking lot: every deployable scheme fails multi-bottleneck max-min; all worsen with N .

scheme	$N=2$	$N=3$	$N=4$	PFC
DCQCN	$1.70\times$	$1.90\times$	$2.32\times$	6–34
TIMELY	$1.83\times$	$1.96\times$	$2.01\times$	72–156
DCTCP	$1.61\times$	$1.72\times$	$1.78\times$	256–280
HPCC	$1.90\times$	$1.92\times$	$1.96\times$	0
RDMA-RCP	$1.003\times$	$1.003\times$	$1.005\times$	0

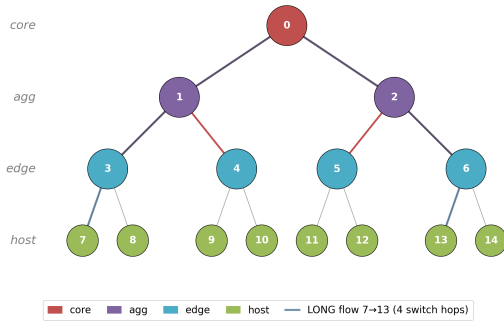
FCT), and the slowest flows are precisely the cross-pod, multi-bottleneck ones stock HPCC starves. We run a 16-flow ring allreduce over all hosts of the $k=4$ fat-tree (4 inter-pod, 4 intra-pod, 8 same-edge segments; 50 MB each) across the same five ECMP hash seeds. RDMA-RCP cuts JCT by 19.5–22.7% (mean 21.4%) at *every* seed, and makes it nearly placement-invariant (17.9–18.0 ms across seeds, versus stock’s 22.3–23.3 ms) — equalized flows cannot be victimized by an unlucky hash. Adding eight 2 MB background flows leaves the JCT gain intact (+20.6% mean) and, in this mix, speeds the background flows up $\sim 3\times$ (0.9–1.3 ms vs 2.2–4.3 ms): under winner-take-all dynamics whether a short flow wins or loses is a placement lottery (they lose here, they won in the WebSearch mix of §5.14), and max-min removes the lottery. Read as a coflow, the 8-flow ECMP workload of §5 shows the same effect: makespan improves 9.8–26.5% across seeds.

5.5 Structurally different topologies

Tree fabric (3-tier, no ECMP). A 3-tier tree with unique shortest paths (15 nodes; 25 Gbps inter-switch, 100 Gbps host; Figure 3a). Stock penalty $1.36\times$, RDMA-RCP $1.003\times$, PFC = 0.

$k = 4$ fat-tree (with ECMP). The canonical fat-tree (20 switches + 16 hosts; 48 links; Figure 3b) with the simulator’s hash-based ECMP routing. Workload: 4 inter-pod flows (5-switch paths) plus 4 intra-pod different-edge flows (3-switch paths). We report two distinct quantities. (i) The *oracle-normalized penalty* — mean long-path slowdown divided by mean short-path slowdown, each measured against the max-min oracle — which removes the effect of ECMP load placement: stock $1.34\times$, RDMA-RCP $1.001\times$ (every contended flow lands at its oracle FCT). (ii) The *raw long/short FCT ratio*, which under RDMA-RCP is still $\sim 1.32\times$ — but this residual is *not* CC unfairness: ECMP hashes some flows onto under-loaded cores so they finish early, inflating the raw ratio independently of the congestion controller. PFC = 0 for both schemes. The point is that, once ECMP load placement is normalized away, RDMA-RCP removes the multi-bottleneck penalty; the path-min fair-rate aggregation is path-invariant, so ECMP requires no design changes. *Hash-seed sensitivity*. Because a single deterministic run fixes one placement, we re-ran the workload across five ECMP hash seeds (an additive `ecmp_seed_offset` knob shifts every switch’s hash seed; the default preserves stock behavior), reporting the *mean*-long/mean-short raw ratio throughout this paragraph. Under stock HPCC that ratio varies $0.97-1.34\times$ across seeds *with no change to the congestion controller* — direct evidence that the raw spread is placement-dominated. At the seed where placement produces real contention

Tree fabric (3-tier, 15 nodes, unique shortest paths — no ECMP)



(a) Tree fabric — unique shortest paths, no ECMP.

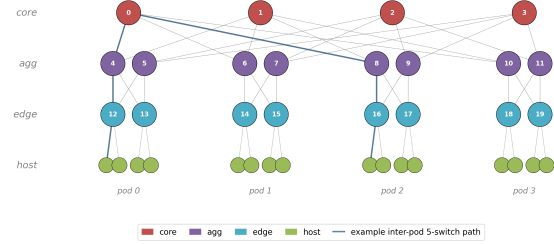
 $k = 4$ ECMP fat-tree: 4 cores, 8 aggs, 8 edges, 16 hosts(b) $k = 4$ ECMP fat-tree (4 pods).

Figure 3: Two structurally different topologies used to test generalization beyond the parking lot. (a) 3-tier tree fabric, 15 nodes; the highlighted line is the long flow $7 \rightarrow 13$, crossing four switches. (b) Canonical $k = 4$ fat-tree (4 cores, 8 aggs, 8 edges, 16 hosts in 4 pods); the highlighted line is a representative inter-pod 5-switch path.

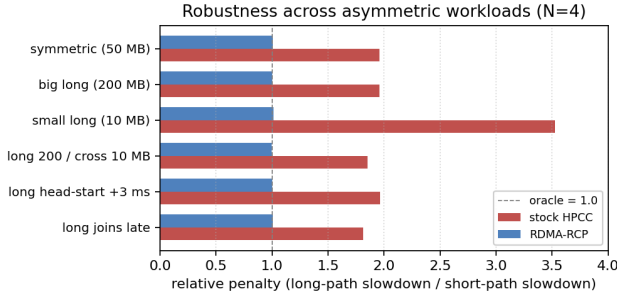


Figure 4: Robustness across asymmetric-size and staggered-start variants at $N = 4$. RDMA-RCP is within 1.0 ± 0.012 across all six scenarios, including the small-long worst case where stock reaches $3.53\times$.

it drops $1.338\times \rightarrow 1.001\times$ under RDMA-RCP. RDMA-RCP is at or below stock at four of five seeds; at seed 202 neither scheme is contended (both ratios < 1 , i.e. the long flows are *avored* by placement) and the two agree within 1.6%. The oracle-normalized conclusion is therefore seed-robust (`run_ecmp_seeds.py` in the artifact).

5.6 Robustness across asymmetric workloads

Across six perturbations (Figure 4), RDMA-RCP holds within 1.0 ± 0.012 . The worst stock case (small multi-bottleneck flow, $3.53\times$ penalty) drops to $1.012\times$.

Start-time jitter. Jittering all five starts by $U[0, 1]$ ms (five seeds): stock $1.96\text{--}1.99\times$, RDMA-RCP $1.008\text{--}1.017\times$. On the $N=1$ control the pairwise spread persists at $1.10\text{--}1.39\times$ across $10\mu\text{s}\text{--}5\text{ms}$ offsets with its *direction* flipping by arrival order — the η -hold lottery, not synchronization — while RDMA-RCP equalizes the pair at every offset.

Table 3: Generalized unfairness (worst-long / best-flow, oracle-normalized) beyond the uniform parking lot. RDMA-RCP holds $\approx 1.0\times$ in every scenario; PFC = 0 throughout.

scenario	stock HPCC	RDMA-RCP
hetero capacities [25,50,25,100] G	$1.82\times$	$1.003\times$
unequal competitors [3,1,2,1]	$1.41\times$	$1.001\times$
two overlapping long flows	$2.87\times$	$1.007\times$
cross-flow churn (arr./dep.)	$2.23\times$	$1.002\times$

5.7 Heterogeneous and dynamic contention

The uniform one-competitor parking lot is deliberately minimal, so we stress its assumptions (Table 3; `run_stress_matrix.py`): heterogeneous capacities, unequal competitor counts, two overlapping multi-bottleneck flows, and cross-flow churn. Each flow is scored against a generic event-driven progressive-filling oracle on the actual topology and schedule; *generalized unfairness* = worst-long / best-flow penalty (each vs. its own oracle FCT), which reduces to the headline ratio in the uniform case. Stock HPCC’s unfairness *grows* with contention richness — worst with two overlapping long flows ($2.87\times$) and churn ($2.23\times$) — while RDMA-RCP holds $1.001\text{--}1.007\times$ throughout, and $1.003\text{--}1.008\times$ under $4\text{--}8\times$ RTT spreads applied to the mode itself; PFC = 0 everywhere. (HPCC’s INT format admits only the discrete rates {25, 50, 100, 200, 400} Gbps; capacity heterogeneity is exercised within that set.)

5.8 Ablation: windows must scale with the fair rate

RDMA-RCP defaults to rate-only operation — the per-flow window cap is disabled (§4). Table 4 shows why, and shows the constraint is narrower than “no windows”: re-enabling HPCC’s `var_win` window degrades the penalty to $1.5\text{--}2.5\times$, worsening with path length, because that window is sized from the NIC line rate rather than the smaller multi-hop bottleneck path. The problem is the window’s *scale*, not windowing itself: a canonical-RCP endpoint that sizes

Table 4: Window ablation: the NIC-scaled window breaks fairness; a fair-rate-scaled canonical RCP window preserves it (all zero PFC).

N	rate-only (default)	NIC-scaled window	$cwnd = R \cdot RTT$
2	1.003×	1.50×	1.004×
3	1.003×	2.00×	1.004×
4	1.005×	2.49×	1.006×

its window from the adopted fair rate, $cwnd = R \cdot \text{baseRTT}$ (the `fs_rcp_window` knob), matches rate-only fairness while bounding in-flight data per flow. Deployments wanting a hard in-flight cap can keep one; it must simply track R , not the NIC.

5.9 Parameter sensitivity

We use the standard RCP defaults ($\alpha = 0.4$, $\beta = 0.226$) untouched. Sweeping each parameter one at a time at $N = 4 - \alpha \in [0.2, 0.6]$, $\beta \in [0.1, 0.4]$, startup $R_0/C \in [0.25, 1.0]$, $\text{minRate} \in [100, 2000]$ Mb/s — the penalty stays within $[1.004, 1.008] \times$ with zero PFC throughout (even $R_0 = C$: the `m_minRate` sender start absorbs the first interval). Within this benchmark family the RCP parameters are not load-bearing; we do not claim the same for arbitrary topologies or joint sweeps. The control interval d tolerates misconfiguration asymmetrically: at $2-4 \times$ too large, or $2 \times$ too small, the penalty stays $1.003-1.011 \times$; at $4 \times$ too small it degrades to $1.212 \times$ (updates outrun the feedback loop) — still far below every baseline, and PFC stays zero throughout.

5.10 Queueing and PFC

For context, every run uses the validated HPCC buffer model: 4 MB shared buffer with dynamic PFC thresholds, ECN marking $k_{\min}/k_{\max}/p_{\max} = 100 \text{ KB}/400 \text{ KB}/0.2$ at 25 Gbps (400/1600 KB at 100 Gbps), 1000 B payload, $1-2 \mu\text{s}$ link delays. With the smoothed startup (§4.2), RDMA-RCP produces **zero PFC events** on every workload in the N -sweep and the robustness matrix. ECN marking remains; queue length is bounded (peak $\approx 46 \text{ KB}$ at $N = 4$, versus $\approx 118 \text{ KB}$ for stock HPCC on the same workload); no flow loss.

5.11 Do-no-harm

The stock `cc_mode 3` smoke tests and N -sweep are byte-identical to the pre-change baseline: the RDMA-RCP additions (implementation name HPCC-FS, `cc_mode 11`) only add code; no existing function’s output for existing inputs is altered.

5.12 HPCC’s home turf: competitive FCT, lower queue

What does the RCP mode cost on *single-bottleneck* workloads — HPCC’s home turf? We measure the validated incast benchmark (3 senders $\rightarrow$ 1 receiver over 25 Gbps; 12 flows, 100–400 KB), sweeping the sender’s startup rate:

Two things stand out (Table 5): RDMA-RCP holds far less queue than HPCC at every startup (38–64 KB vs 143 KB), and HPCC’s only lead — FCT at the conservative 1 Gbps startup ($\sim 15\%$) — closes and reverses at 8 Gbps (242 vs 259 μs at one-quarter the queue, zero

Table 5: HPCC’s home turf (single-bottleneck incast). RDMA-RCP holds far less queue than HPCC at every startup, and a moderate startup also matches or beats HPCC’s FCT. The same startup leaves the multi-bottleneck fairness headline unchanged ($1.003-1.004 \times$, PFC = 0).

	mean FCT	tail FCT	peak queue
HPCC	258.6 μs	459.8 μs	143 KB
RDMA-RCP (startup 1 Gbps)	297.5 μs	490.3 μs	64 KB
RDMA-RCP (startup 5 Gbps)	271.2 μs	464.3 μs	49 KB
RDMA-RCP (startup 8 Gbps)	242.0 μs	416.8 μs	38 KB

PFC). The 8 Gbps point does not disturb the multi-bottleneck headline ($1.003-1.004 \times$; robustness $1.002-1.009 \times$; PFC = 0) — a second operating point; the headline runs use 1 Gbps. So on the home-turf workloads we measured, RDMA-RCP’s fairness mode comes at *little-to-no FCT cost and a queue benefit* — not a trade-off. Worst-case in-flight data is also bounded in rate-only operation: before the first fair-rate ACK a sender emits at most $\text{startup} \times \text{baseRTT}$ (1 Gbps $\times 21.4 \mu\text{s} \approx 2.7 \text{ KB}$ per sender; even 1000 synchronized senders inject $\sim 2.7 \text{ MB}$ in the first RTT), and the $cwnd = R \cdot RTT$ variant (§5.8) restores a hard per-flow cap thereafter.

We are careful not to overclaim: RDMA-RCP still lacks HPCC’s per-hop, window-bounded reaction, so on rapidly-varying or adversarial-microburst workloads (which we did not evaluate) HPCC’s precision may matter; recovering that while keeping the fairness mode is what the hybrid overlay of §7 targets.

5.13 Per-flow rate trace, before vs after

The before-vs-after rate trace (Figure 5) is the cleanest qualitative result. Stock HPCC: the long flow collapses to $\sim 0.4 \text{ Gbps}$ while the four cross flows hold $\sim 23 \text{ Gbps}$ each for the full 19 ms contention; RDMA-RCP: all five flows converge to $\sim 12.5 \text{ Gbps}$ within $\sim 4 \text{ ms}$ and stay locked at fair share.

5.14 Scope of the headline claim

The $\approx 1.0 \times$ headline is a statement about the *congestion-control component* of multi-bottleneck unfairness on contention benchmarks — not a claim of universal FCT improvement on production workloads. Four first-class limits. (i) *Mixed short-flow workloads* do not exhibit the penalty we target: under a WebSearch distribution stock HPCC’s oracle-normalized penalty is already $1.001 \times$ (RDMA-RCP: $0.581 \times$), and on mean slowdown RDMA-RCP trades $\sim 15\%$ off multi-bottleneck flows for $\sim 45\%$ on short paths (a mean dominated by sub-10 KB flows). Short-flow-FCT workloads [4, 18] may prefer stock HPCC; max-min matters when multi-bottleneck flows gate a job (§5.4) — a per-class mode selection, not a global replacement. (ii) *Fabric scale*: at $k = 6, 8$ our scaling workload does not form persistent inter-pod contention (both schemes’ penalty < 1), so the mode is exercised, but untested, where large-fabric contention would arise; a saturating workload is future work. (iii) *ECMP placement*: a raw $\sim 1.3 \times$ long/short spread from hash placement remains; RDMA-RCP removes the CC component only. (iv) *Coexistence*: the modes do not share a link fairly even under DRR reservation (§4.3); RDMA-RCP is a fabric-wide mode.

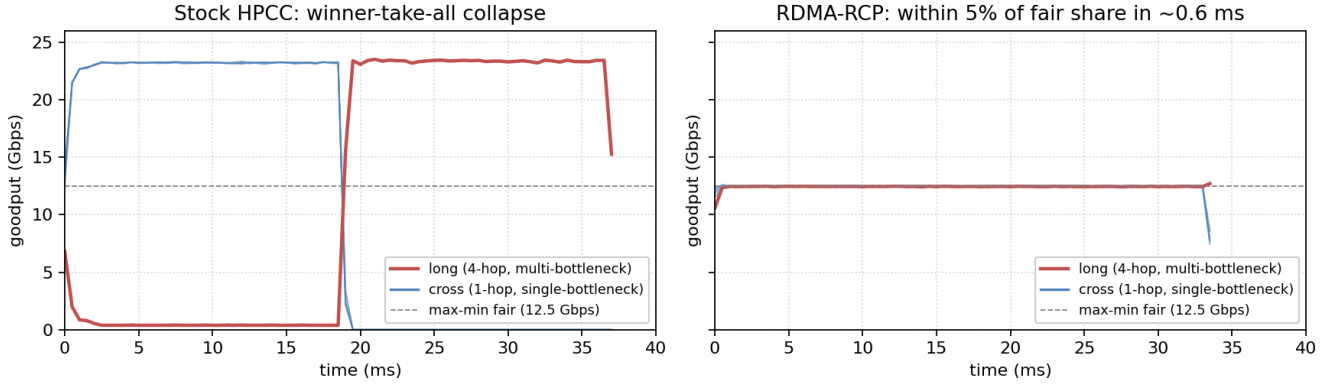


Figure 5: Per-flow goodput at $N = 4$, stock HPCC (left) vs RDMA-RCP (right). Stock collapses the long flow to ~ 0.4 Gbps while cross flows hold ~ 23 Gbps for 18 ms (winner-take-all). RDMA-RCP converges all five flows to within 5% of the max-min fair share (12.5 Gbps) in ~ 0.6 ms and to within 1% by ~ 2.3 ms.

6 Related Work

Datacenter transport. DCTCP [3], DCQCN [25], TIMELY [17], and Swift [14] trade buffering for latency via ECN- and delay-based control; Homa [18] and pFabric [4] prioritize short flows; HPCC [15] drives control from per-hop INT, with On-Ramp [16] and PowerTCP [1] extending the precise-signal philosophy. Across this line the target is single-bottleneck FCT, tail latency, or queueing; multi-bottleneck fairness receives little attention — and §5.3 shows every member of this family we evaluate fails it.

In-network telemetry. INT [13] lets the data plane stamp per-hop state into packets on programmable pipelines, and HPCC was among the first to drive congestion control directly from it. The cost is header overhead, which later work minimizes: HPCC-PINT [24] compresses the per-hop record into a single probabilistic field, and Poseidon [23] restructures INT-based control for deployability on commodity hardware. RDMA-RCP continues this minimal-telemetry direction but changes *what* the field carries: not a raw load sample to be interpreted by the sender, but an explicit fair *rate* computed at the switch and min-aggregated along the path—one 64-bit field, like PINT, yet a decision rather than a measurement.

Explicit-rate and switch-assisted control. Having the switch compute and return a sending rate is a classic idea. ATM ABR explicit-rate schemes such as ERICA [11] computed a fair share per port decades ago; XCP [12] decomposed switch feedback into separate efficiency and fairness controllers; and RCP [9] reduced this to a single fair-rate scalar per link with no per-flow state, packet-stamped, with senders taking the path minimum. RCP is the closest mechanism to ours: RDMA-RCP adopts its update rule but situates it as a *minimal fairness signal injected into an existing INT load-signaling system* rather than a standalone protocol, keeping HPCC’s single-rate sender and per-hop transport. RDMA-RCP’s per-port update and path-min aggregation *are* RCP’s; the endpoint differs (rate-only by default, though the canonical window $cwnd = R \cdot RTT$ performs equally, Table 4), the control interval is a fabric constant, updates are event-driven, and the port applies $R_0 = C/2$ and a rate floor.

What RDMA-RCP adds over deploying RCP as a separate protocol is packaging, not control: one field on the INT transport the datacenter already runs, a byte-identical stock HPCC path, and per-cc_mode selection — no second protocol stack.

In-switch fairness. Fair queueing [8], DRR [20], CSFQ [22], and AFD [19] enforce fairness by per-packet queueing or dropping; programmable schedulers [21] realize such policies at line rate. RDMA-RCP instead *advertises* one fair rate per port for rate-controlled senders to adopt — no per-flow tracking, no extra queues — implementable on the match-action pipelines [7] HPCC’s INT already assumes.

Closely related switch-assisted DC schemes. Bolt [5] accelerates HPCC’s loop with sub-RTT per-flow switch assistance, targeting tail latency and ramp-up — orthogonal to our objective: its feedback carries load/supply, not a fair share, so the shared-reference argument of §3 applies to its senders too; a faster load-signal loop converges faster to the same equilibrium. A head-to-head would measure reaction speed, not the placement question this paper isolates; we leave it to future work.

Fairness foundations. Max-min, progressive filling, and the parking lot are classical [6]; our contribution is the measurement, not a new definition.

7 Discussion

Relation to HPCC, and a hybrid path. RDMA-RCP inherits RCP’s character — fair, low-queue — without HPCC’s single-flow precision (§5.12). A future hybrid runs HPCC unchanged with the fair rate as a per-flow ceiling, $\min(\text{rate}_{\text{HPCC}}, \text{fairRate})$; the obstacle is HPCC’s NIC-sized window (§5.8 suggests a fair-rate-scaled one).

INT hop limit. Stock HPCC’s per-hop INT cost grows with path length (reference cap $\text{maxHop}=5$, corrupting control input past $N=4$); RDMA-RCP’s one-field format is hop-count-independent ($\approx 1.0\times$ at $N = 5, 6$).

Scaling and hardware. At $k = 6, 8$ the fabric's $(k/2)^2$ cores spread inter-pod flows so persistent contention does not materialize for our scaling workload (both schemes' penalty < 1 ; RDMA-RCP adds no overhead, 0 PFC) — a saturating workload is future work. (Capacity, competitor, and RTT heterogeneity are now exercised directly in §5.7.) The path-min aggregation is path-invariant under ECMP; residual FCT spread is load placement, a routing issue. On hardware we offer a mapping, not a prototype: three registers per port (R , timestamp, byte snapshot); a dequeue-triggered fixed-point update with $1/C$ and β/d precomputed (two multiplies, two subtractions, a clamp); and a 64-bit compare-and-write path-min per packet. Each piece has P4 precedent, but stage budget and dequeue-time queue-depth access are switch-specific; a Tofino prototype remains future work.

Limitations. Three topology families; in-sim baselines only (DC-CCN, TIMELY, DCTCP, multi-rate HPCC, PINT) — no Bolt/PowerTCP re-implementation and no testbed; per-class coexistence needs a reserved share we do not design (§4.3); the ECMP study is a small controlled workload (though seed-swept), not a realistic mixed-size + ECMP combination. All numbers come from one simulator — but its control logic is the original HPCC implementation with interface-level modernization only, verified end-to-end by reproducing the original baselines, and the $N \leq 4$ regime sits within the 5-hop INT budget by construction; a belt-and-braces run on the unmodified legacy build remains future work. We did, however, verify that the result is insensitive to the RCP parameters (§5.9), so the absence of tuning is not a hidden confound.

8 Conclusion

Multi-bottleneck unfairness spans every deployable RDMA scheme in our stack, and HPCC's instance is a stable, RTT-independent, winner-take-all equilibrium ($\sim 2\times; 3.5\times$ for small flows; unchanged over $10\times$ longer backlogs). Six sender-side corrections fail — including an exact private mirror of RCP — and a controlled fluid experiment shows why: with identical observations, law, and epochs, private state still freezes arrival-order unfairness; max-min needs one *shared* reference, and a per-port register is its cheapest home. RDMA-RCP carries that register's value in one INT field — one fair-rate register (three words) per port — reaching penalty $1.005\times$ with zero PFC across every workload we tested, rate-only or canonically windowed, converging sub-millisecond and independent of path length, with the stock HPCC code path byte-for-byte unchanged. Sharing an unreserved link with stock HPCC, however, lets the FS class capture bandwidth (§4.3): a scheduler reservation alone does not suffice — the mode belongs on sliced telemetry or a separate fabric, not in free competition.

References

- [1] Vamsi Addanki, Maria Apostolaki, Manya Ghobadi, Stefan Schmid, and Laurent Vanbever. 2022. PowerTCP: Pushing the Performance Limits of Datacenter Networks. In *Proc. USENIX NSDI*. 51–70.
- [2] Alibaba. 2019. HPCC reference simulator (ns-3). <https://github.com/alibaba-edu/High-Precision-Congestion-Control>. int-header.h: static const uint32_t maxHop = 5.
- [3] Mohammad Alizadeh, Albert Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. 2010. Data Center TCP (DCTCP). In *Proc. ACM SIGCOMM*. 63–74.
- [4] Mohammad Alizadeh, Shuang Yang, Milad Sharif, Sachin Katti, Nick McKeown, Balaji Prabhakar, and Scott Shenker. 2013. pFabric: Minimal Near-Optimal Datacenter Transport. In *Proc. ACM SIGCOMM*.
- [5] Serhat Arslan, Yuliang Li, Gautam Kumar, and Nandita Dukkkipati. 2023. Bolt: Sub-RTT Congestion Control for Ultra-Low Latency. In *Proc. USENIX NSDI*. 17 pages.
- [6] Dimitri Bertsekas and Robert Gallager. 1992. *Data Networks* (2nd ed.).
- [7] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. 2014. P4: Programming Protocol-Independent Packet Processors. *ACM SIGCOMM Computer Communication Review* 44, 3 (2014).
- [8] Alan Demers, Srinivasan Keshav, and Scott Shenker. 1989. Analysis and Simulation of a Fair Queueing Algorithm. In *Proc. ACM SIGCOMM*.
- [9] Nandita Dukkkipati. 2008. *Rate Control Protocol (RCP): Congestion Control to Make Flows Complete Quickly*. Ph.D. Dissertation. Stanford University.
- [10] Nandita Dukkkipati and Nick McKeown. 2006. Why Flow-Completion Time is the Right Metric for Congestion Control. *ACM SIGCOMM Comput. Commun. Rev.* 36, 1 (2006), 59–62.
- [11] Shivkumar Kalyanaraman, Raj Jain, Sonia Fahmy, Rohit Goyal, and Bobby Vandalore. 2000. The ERICA Switch Algorithm for ABR Traffic Management in ATM Networks. *IEEE/ACM Transactions on Networking* 8, 1 (2000).
- [12] Dina Katabi, Mark Handley, and Charlie Rohrs. 2002. Internet Congestion Control for Future High Bandwidth-Delay Product Environments. In *Proc. ACM SIGCOMM*. 89–102.
- [13] Changhoon Kim, Anirudh Sivaraman, Naga Katta, Antonin Bas, Advait Dixit, and Lawrence J. Wobker. 2015. In-band Network Telemetry via Programmable Dataplanes. In *Proc. ACM SIGCOMM (Industrial Demo)*.
- [14] Gautam Kumar, Nandita Dukkkipati, Keon Jang, Hassan M. G. Wassel, Xian Wu, Behnam Montazeri, Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael Ryan, David Wetherall, and Amin Vahdat. 2020. Swift: Delay is Simple and Effective for Congestion Control in the Datacenter. In *Proc. ACM SIGCOMM*.
- [15] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, and Minlan Yu. 2019. HPCC: High Precision Congestion Control. In *Proc. ACM SIGCOMM*. 44–58.
- [16] Shiyu Liu, Ahmad Ghalayini, Mohammad Alizadeh, Balaji Prabhakar, Mendel Rosenblum, and Anirudh Sivaraman. 2021. Breaking the Transience-Equilibrium Nexus: A New Approach to Datacenter Packet Transport. In *Proc. USENIX NSDI*.
- [17] Radhika Mittal, Vinh The Lam, Nandita Dukkkipati, Emily Blem, Hassan Wassel, Monia Ghobadi, Amin Vahdat, Yaogong Wang, David Wetherall, and David Zats. 2015. TIMELY: RTT-based Congestion Control for the Datacenter. In *Proc. ACM SIGCOMM*. 537–550.
- [18] Behnam Montazeri, Yilong Li, Mohammad Alizadeh, and John Ousterhout. 2018. Homa: A Receiver-Driven Low-Latency Transport Protocol Using Network Priorities. In *Proc. ACM SIGCOMM*.
- [19] Rong Pan, Lee Breslau, Balaji Prabhakar, and Scott Shenker. 2003. Approximate Fairness through Differential Dropping. *ACM SIGCOMM Computer Communication Review* 33, 2 (2003).
- [20] M. Shreedhar and George Varghese. 1995. Efficient Fair Queueing using Deficit Round Robin. In *Proc. ACM SIGCOMM*.
- [21] Anirudh Sivaraman, Suvinay Subramanian, Mohammad Alizadeh, Sharad Chole, Shang-Tse Chuang, Anurag Agrawal, Hari Balakrishnan, Tom Edsall, Sachin Katti, and Nick McKeown. 2016. Programmable Packet Scheduling at Line Rate. In *Proc. ACM SIGCOMM*.
- [22] Ion Stoica, Scott Shenker, and Hui Zhang. 1998. Core-Stateless Fair Queueing: Achieving Approximately Fair Bandwidth Allocations in High Speed Networks. In *Proc. ACM SIGCOMM*.
- [23] Weitao Wang, Masoud Moshref, Yuliang Li, Gautam Kumar, T. S. Eugene Ng, Neal Cardwell, and Nandita Dukkkipati. 2023. Poseidon: Efficient, Robust, and Practical Datacenter CC via Deployable INT. In *Proc. USENIX NSDI*.
- [24] Liron Schiff Yang, Ran Ben Basat, Michael Mitzenmacher, Maria Apostolaki, Aviv Klein, Erez Saito, and Avi Mendelson. 2020. PINT: Probabilistic In-band Network Telemetry. In *Proc. ACM SIGCOMM*. 662–677.
- [25] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. 2015. Congestion Control for Large-Scale RDMA Deployments. In *Proc. ACM SIGCOMM*. 523–536.